GUIA VISUAL 030

JOBS, PLANOS NOHUP & DISOWN

A varredura de seis horas some junto com a conexão SSH — por causa de um sinal só.

030
GUIAS LINUX

A SESSÃO: QUEM RECEBE O `CTRL-C` E QUEM LEVA O `SIGHUP`

`/dev/pts/0`
terminal de controle

→

bash
líder da sessão
PID = SID = 1342

→

- **1º plano · `vim`** único que lê o teclado e recebe **`Ctrl-C`** / **`Ctrl-Z`**
- **`%1` · `tar czf … &`** escreve na tela, mas *ler* dela o para com **`SIGTTIN`**
- **`%2` · `nmap … & job = pipeline inteiro`** = um grupo de processos

Ao cair a conexão, o kernel manda **SIGHUP** ao líder da sessão, e o bash repassa a *todos* os jobs — ação padrão: terminar. Sair digitando `exit` no bash **não** faz isso (opção `huponexit` desligada); o `zsh` do Kali se comporta diferente — não conte com ele.

O CICLO DO JOB

- ✓ `cmd &` inicia em 2º plano e devolve o prompt: `[1] 2847`
- ✓ **Ctrl-Z** envia `SIGTSTP` : estado **T**, memória intacta, zero CPU.
- ✓ `bg` retoma em 2º plano · `fg` traz de volta ao 1º.
- ✓ `jobs -l` mostra PID; **+** é o job corrente, **-** o anterior.
- ✓ *Stopped (tty input)* = o job pediu teclado: `fg` , responda, **Ctrl-Z**, `bg` .

REFERENCIAR E SINCRONIZAR

%1 número · **%+** ou **%%** corrente · **%-** anterior

%vim começa com · **%?relato** contém

`kill %3` sinaliza o **job** 3 (o grupo inteiro).

`$!` guarda o PID do último `&` ; `wait` espera e devolve o `$?` real.

`t1 & t2 & t3 & wait` — o paralelismo básico do shell.

jobs não atravessa terminais: entre painéis do tmux, use PID com `pgrep` .

DESACOPLAR DO TERMINAL	QUANDO SE USA	O QUE ACONTECE COM A SAÍDA
`nohup cmd > log 2>&1 &`	Antes de iniciar	Ignora o SIGHUP; sem redirecionar, despeja em `nohup.out`
`disown -h %1`	Depois de iniciar — "esqueci o nohup"	Tira o job da tabela do shell; a saída no terminal **se perde**
`setsid cmd < /dev/null`	Antes — desassociar de vez	Sessão nova, `TTY ?` ; exige redirecionamento
`systemd-run --user --unit=x`	Quando o certo é ser serviço	Vira unidade: log no `journald` , controle por `systemctl`
`tmux` / `screen`	Tarefa longa **e interativa**	**Preservada e reconectável** — você volta e vê a tela (ficha 031)

`kill %1` **e** `kill 1` **não são a mesma coisa.** Com `%` o argumento é número de *job* do shell; sem `%` é um **PID** — e o PID 1 é o `systemd` . Como root, `kill -9 1` é o caminho mais curto para um sistema que só volta com reset físico. O `%` não é decoração.

RESUMO RÁPIDO

- ✓ **Job é o pipeline inteiro** = um grupo de processos; por isso o `Ctrl-C` derruba todos os estágios.
- ✓ `jobs` / `fg` / `bg` são internos do shell e valem só naquele terminal.
- ✓ Terminal fechou → `SIGHUP` ao líder da sessão → repassado aos jobs.
- ✓ `nohup`, `disown` e `setsid` mantêm vivo; **nenhum preserva a interação**.
- ✓ Sem tmux, dá para espiar: `tail -f /proc/PID/fd/1`; em emergência, `reptyr PID` rouba o processo.

NO KALI

Ferramenta longa merece **tmux**, não `nohup`: você quer *ver a tela* ao reconectar. Inicie o `nmap` com `-oA scan` — sem arquivo de saída não existe `--resume`, e a interrupção obriga a recomeçar. *Listener* do metasploit ou `nc -lvnp` preso ao terminal morre com o SSH e derruba quem estiver conectado. E `nohup.out` esquecido no alvo é **vestígio**: redirecione para a sua área e limpe no encerramento. Para caçar persistência: `ps -eo pid,ppid,tty,comm | awk '$3=="?"'`.